**Institut National des Postes et Télécommunication**
Conteneurisation des Applications – ASEDS
Prof. Driss ALLAKI
Année universitaire 2026/2027

# Devoir 2
## Déploiement Automatisé et Mise à l'Échelle d'une Application Web sur un Cluster d'Orchestration de Conteneurs

Enoncé

## I. Prérequis

Réalisation du Devoir 1 (Conteneurisation d'une application Web).

## II. Contexte

Forts du succès de la phase de conteneurisation avec Docker, les équipes de NeoTech ont pu valider le PoC et résoudre les problèmes d'incompatibilité d'environnements entre les postes de développement, de test et de production.

Cependant, face à la croissance rapide de l'entreprise et au succès potentiel de leurs applications, la gestion manuelle des conteneurs isolés (via Docker Compose) montre rapidement ses limites en matière de scalabilité, de haute disponibilité, de résilience et d'automatisation des déploiements.

Pour franchir une nouvelle étape, NeoTech souhaite désormais industrialiser l'infrastructure de production. En tant que continuité de votre stage, il vous est demandé de reprendre l'application web 3-tiers *Travelo* (React, Spring Boot, MySQL) préalablement conteneurisée et de concevoir son déploiement au sein d'un cluster d'orchestration de conteneurs (Kubernetes).

Ce nouveau jalon vise à automatiser le déploiement, à garantir la scalabilité dynamique des services et à assurer une tolérance aux pannes indispensable pour la mise en production de la solution.

## III. Travail à réaliser

**Déploiement de l'application Travelo dans un cluster Kubernetes en local**

1. Créer un cluster Kubernetes local avec Minikube ou équivalent.
2. Configurer un namespace nommé *travelo*.
3. Ajouter les fichiers Kubernetes dans un nouveau repo GitLab (ou GitHub), puis exécuter les dans le cluster K8s local (au sein du namespace *travelo*).

    a. Créer les Kubernetes manifests (fichiers YAML) nécessaires pour déployer cette application dans un cluster Kubernetes (sous un namespace nommé *travelo*).
        - Deployment
        - Service
        - ConfigMap
        - Secret
        - Statefulset

NB1 : Vous avez le choix d'utiliser n'importe quel registre d'images (docker hub, harbor, registre privée en local, etc.) pour récupérer les images nécessaires.

NB2 : Créer 2 réplicas pour chaque partie de l'application (frontend et backend)

b. Définir des quotas pour la consommation des ressources Mémoire et CPU de chaque Pod exécuté dans le namespace « travelo ».

c. Contrôler l'accès aux ressources existantes dans le namespace « travelo » par un rôle RBAC.

d. Spécifier un budget de perturbation (Disruption Budget) pour votre application.

e. Créer et configurer pour cette application un :

- Liveness Prob
- Readiness Prob
- Startup Prob

NB : Vérifier les événements générés par les différents pods de l'application.

f. Créer un Ingress pour accéder à l'application depuis un navigateur moyennant un nom de domaine.

4. S'assurer en local avec des commandes kubectl de la création des objets/contrôleurs Kubernetes convenables.

   S'assurer que l'application a été bien conteneurisée et qu'elle fonctionne correctement. (Prendre des prises d'écran du navigateur)

**Bonus**

**A.** Repenser à créer et gérer la partie base de données de l'application en utilisant le concept de : *Operator*. (Créer un cluster de bases de données)

**B.** Utiliser l'outil *Skaffold* pour détecter les changements de source pendant le développement, construire et créer automatiquement les images Docker et puis pour pousser et déployer les artefacts dans le cluster Kubernetes.

**C.** Créer "from scratch" un/des *Helm* chart(s) pour cette application. Publier votre chart dans un repo public, puis déployer l'application via Helm (sous un autre namespace nommé "*Helm*").

## IV. Consignes et Rendu

- Ce travail est à réaliser **individuellement.**
- Le rendu de ce travail est sous forme d'un **document au format PDF** qui doit inclure :
  - Des prises d'écrans détaillées des différentes étapes suivies pour réaliser le travail.
  - Des descriptions textuelles du travail.